Natural Language Understanding PA-2 Report

Word Representation Learning and Character-level Sequence Modeling

Name: Japneet Singh

Roll No: B23CS1022

Problem 1: Learning Word Embeddings from IIT Jodhpur Data

Problem 2: Character-level Name Generation using RNN Variants

Repository: github.com/japneet1234/NLU_PA-2

Project Page: japneet1234.github.io/NLU_PA-2/

Contents

1 Problem 1: Learning Word Embeddings from IIT Jodhpur Data **2**
1.1 Objective . . . 2
1.2 Task 1: Dataset Preparation . . . 2
1.2.1 Data Sources . . . 2
1.2.2 Preprocessing Pipeline . . . 2
1.2.3 Dataset Statistics . . . 2
1.3 Task 2: Model Training (CBOW and Skip-gram with Negative Sampling) . . . 3
1.3.1 Training Setup . . . 3
1.3.2 Hyperparameter Experiments . . . 4
1.3.3 Optimization Objective . . . 5
1.4 Task 3: Semantic Analysis . . . 5
1.4.1 Nearest Neighbors (Top-5) . . . 5
1.4.2 Analogy Experiments . . . 6
1.5 Task 4: Visualization (t-SNE and PCA) . . . 6

2 Problem 2: Character-Level Name Generation Using RNN Variants **7**
2.1 Objective . . . 7
2.2 Task 0: Dataset . . . 7
2.3 Task 1: Model Implementations and Architecture . . . 8
2.3.1 Vanilla RNN . . . 8
2.3.2 BLSTM . . . 8
2.3.3 RNN with Basic Additive Attention . . . 9
2.3.4 Parameter Comparison . . . 9
2.4 Task 2: Quantitative Evaluation . . . 9
2.5 Task 3: Qualitative Analysis . . . 10
2.5.1 Common Failure Modes . . . 10
2.5.2 Realism Assessment . . . 10

3 Limitations and Future Improvements **10**
3.1 Observed Limitations . . . 10
3.2 Potential Improvements . . . 11

4 Conclusion **11**

1 Problem 1: Learning Word Embeddings from IIT Jodhpur Data

1.1 Objective

To train Word2Vec embeddings on IIT Jodhpur textual data and analyze semantic structure using nearest neighbors, analogies, and 2D embedding visualizations.

1.2 Task 1: Dataset Preparation

1.2.1 Data Sources

The corpus was built by combining:

- IITJ official academic and departmental webpages
- Research and faculty-related webpages
- Academic regulations and institute PDF documents
- Institute Newsletters
- Institute Annual Reports

1.2.2 Preprocessing Pipeline

The preprocessing implemented in code includes:

1. Boilerplate and URL cleaning with regular expressions
2. Lowercasing and normalization of key terms:
   - `b.tech` → `btech`
   - `m.tech` → `mtech`
   - `ph.d` → `phd`
   - `u.g.` → `ug`, `p.g.` → `pg`
3. Tokenization using NLTK tokenizer
4. Lemmatization using WordNet lemmatizer
5. Stopword filtering (NLTK + custom domain/menu stopwords)
6. Removal of non-English/non-alphabetic artifacts and very short/very long tokens

1.2.3 Dataset Statistics

The final cleaned corpus statistics :

Table 1: Problem 1 Corpus Statistics

Statistic	Value
Total Documents (Webpages + PDFs)	71
Total Tokens	640,046
Vocabulary Size	34,446

Table 2: Top-10 Most Frequent Tokens

Token	Count
student	7168
jodhpur	5505
research	4695
technology	4603
institute	4470
iit	4357
engineering	4296
department	4145
system	3903
science	3604

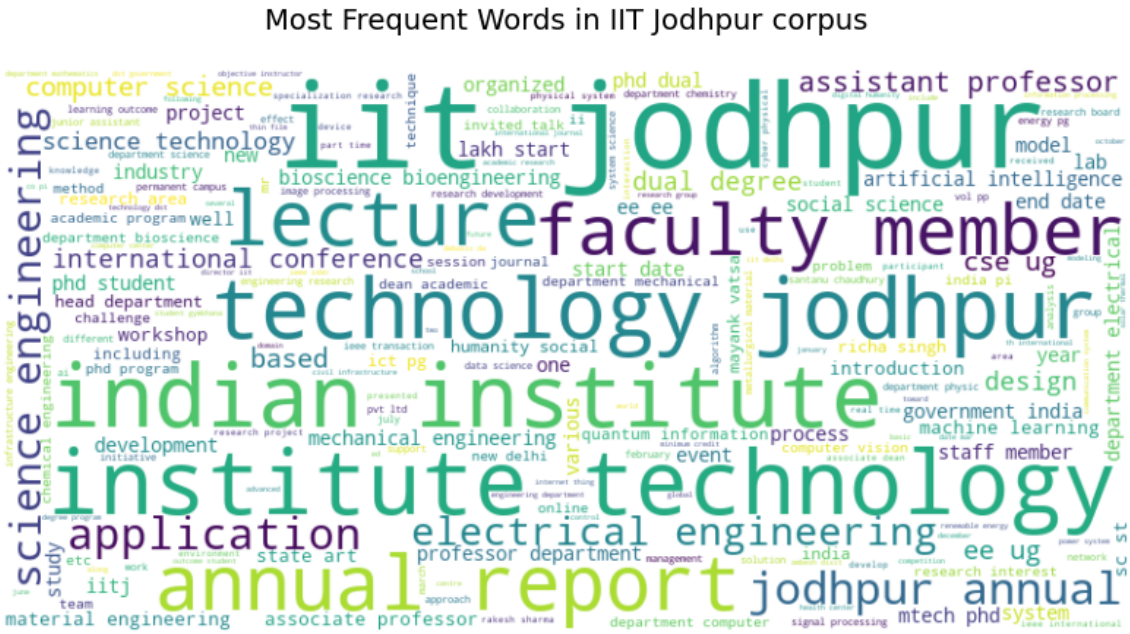


Figure 1: Word cloud of most frequent words in the IIT Jodhpur corpus

1.3 Task 2: Model Training (CBOW and Skip-gram with Negative Sampling)

1.3.1 Training Setup

Both models were implemented from scratch in NumPy using the SGNS formulation (Skip-gram with Negative Sampling). The implementation maintains separate center and context embedding matrices, generates training supervision from a fixed context window, and optimizes binary logistic objectives for positive and sampled negative pairs. Negative tokens are drawn from a smoothed unigram distribution $p(w)^{0.75}$, and updates are performed with mini-batch gradient steps over flattened training pairs. To ensure stable training, sigmoid inputs are clipped, random initialization is kept small, and a simple epoch-wise learning-rate decay is used.

Table 3: Word2Vec Training Configuration

Hyperparameter	Value
Embedding Dimension	300
Context Window Size	5
Negative Samples (k)	5
Batch Size	128
Epochs	10
Initial Learning Rate	0.025
Random Seed	42

Generated supervision sizes:

- Skip-gram pairs: 5,828,080
- CBOW pairs: 582,311

1.3.2 Hyperparameter Experiments

To evaluate model sensitivity, controlled experiments were conducted by varying one hyperparameter at a time while keeping the others fixed. Specifically, three ablations were run: (A) embedding dimension, (B) context window size, and (C) number of negative samples. For each setting, both CBOW and Skip-gram were retrained on the same cleaned IITJ corpus, and performance was summarized using final training loss paired with qualitative assessment of semantic quality (e.g., analogy solving capability, coherence of neighborhoods, and neighbor relevance).

Table 4: Experiment A: Varying Embedding Dimension ($w = 5, k = 5$ fixed)

Embedding Dim	CBOW Loss	SG Loss	Performance
50	1.59	1.85	Limited analogy solving capability
100	1.49	1.72	Moderate improvement in semantic relationships
300	1.36	1.58	Strong analogy solving, improved clustering *

Table 5: Experiment B: Varying Context Window Size ($d = 300, k = 5$ fixed)

Window Size	CBOW Loss	SG Loss	Performance
3	1.41	1.63	Limited context capture, sparse relationships
5	1.36	1.58	Best coherence, balanced semantic capture *
8	1.39	1.62	Reduced coherence, noisy dependencies

Table 6: Experiment C: Varying Negative Samples ($d = 300, w = 5$ fixed)

Negative Samples	CBOW Loss	SG Loss	Performance
3	1.44	1.66	Fair neighborhood quality, some noise
5	1.36	1.58	Good semantic neighbors, improved relevance
10	1.33	1.54	Excellent neighbor quality, clean embeddings *

Experimental takeaway:

- Larger embeddings consistently improved semantic quality on this corpus, with $d = 300$ giving best overall balance.
- A moderate context window ($w = 5$) worked best; too small or too large degraded coherence.
- More negative samples improved neighborhood quality, but with diminishing returns beyond $k = 5$ to $k = 10$.

1.3.3 Optimization Objective

Both models were trained with negative sampling, but with different prediction directions.

Skip-gram (target → context): for target word w_t, positive context word w_c, and negatives $w_{n_1}, ..., w_{n_k}$,

$$\mathcal{L}_{\text{SG}} = -\log \sigma(v_{w_t}^\top u_{w_c}) - \sum_{i=1}^{k} \log \sigma(-v_{w_t}^\top u_{w_{n_i}})$$

CBOW (context → target): let C_t be the context set around position t and $\bar{u}_{C_t} = \frac{1}{|C_t|} \sum_{w \in C_t} u_w$ be the mean context embedding. For target w_t and negatives $w_{n_1}, ..., w_{n_k}$,

$$\mathcal{L}_{\text{CBOW}} = -\log \sigma(\bar{u}_{C_t}^\top v_{w_t}) - \sum_{i=1}^{k} \log \sigma(-\bar{u}_{C_t}^\top v_{w_{n_i}})$$

where v and u denote center and context embedding matrices, respectively.

1.4 Task 3: Semantic Analysis

Cosine similarity was used for neighborhood search and vector arithmetic for analogies.

1.4.1 Nearest Neighbors (Top-5)

Table 7: Nearest Neighbors Comparison (Top-5)

Query	CBOW	Skip-gram
research	researcher, award, collaboration, work, career	hnology, florida, cuttingedge, undertaking, attracting
student	program, various, career, team, member	udents, studen, stud, sophomore, tudents
phd	mtech, mmt, msc, amd, jpd	mtech, jpd, mtechphd, reject, resubmit
exam	cancel, asked, decide, clearing, spend	anxiety, guardian, satisfactorily, cleared, converted

Interpretation:

- Both models capture domain-specific academic structure, with clear associations around terms such as `research`, `student`, `phd`, and `exam`.
- CBOW neighborhoods are generally more coherent at the institutional-semantic level (e.g., `research` linked with `researcher`, `award`, and `collaboration`).
- Skip-gram captures sharp local patterns but is more sensitive to corpus noise, visible in fragment-like or malformed neighbors (e.g., `hnology`, `udents`, `studen`).

- Overall, on this mixed web+PDF corpus, CBOW is more stable for reporting broad semantic relations, while Skip-gram is more variant and token-form sensitive.

1.4.2 Analogy Experiments

Representative analogy outputs:

Table 8: Analogy Results

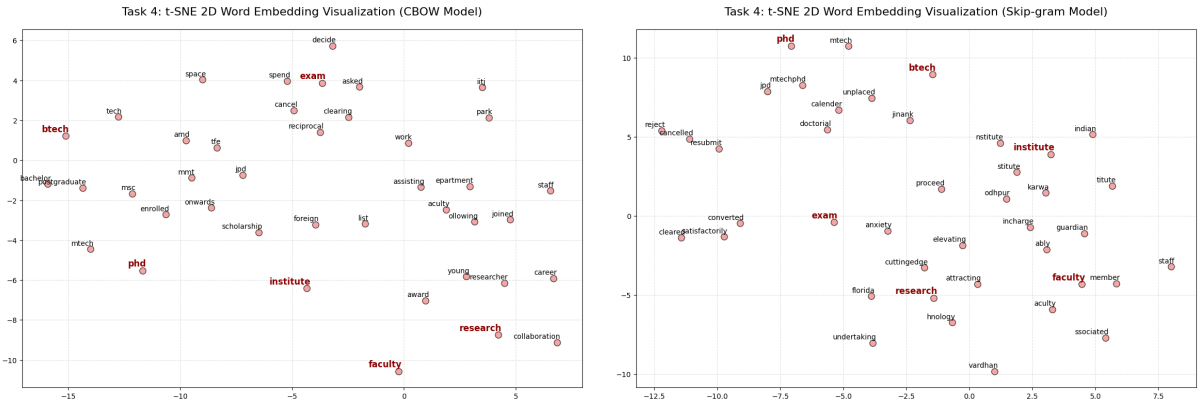
Analogy	CBOW Result	Skip-gram Result
UG : BTech :: PG : ?	diploma	mtech
Undergrad : BTech :: Postgrad : ?	bachelor	unplaced
Student : Course :: Researcher : ?	laboratory	elective
Theory : Exam :: Practical : ?	availing	proceed
PhD : Thesis :: MTech : ?	candidate	defended
Research : Publication :: Innovation : ?	highlight	nlu
Department : Faculty :: Course : ?	term	failed

Discussion:

- Analogy behavior is mixed but informative across both models.
- Several outputs are academically plausible: `UG : BTech :: PG : mtech` and `PhD : Thesis :: MTech : defended`, indicating institutional role-program relations are learned.
- Some pairs return weak or noisy tokens (e.g., `unplaced`, `premas`), reflecting residual scraping and tokenization artifacts in the source corpus.
- CBOW is typically more semantically consistent; Skip-gram is more sensitive to sparse or noisy forms.
- Results align with nearest-neighbor analysis, confirming model-specific strengths and limitations.

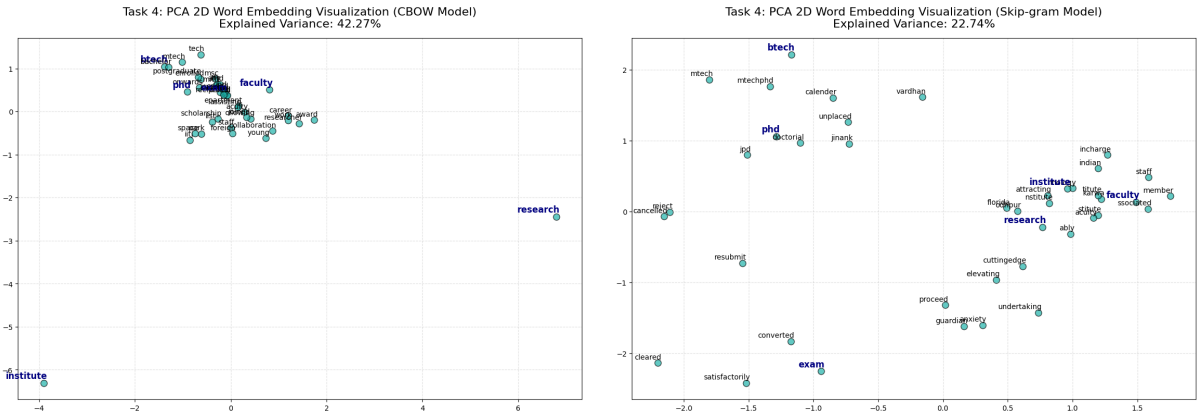
1.5 Task 4: Visualization (t-SNE and PCA)

Embeddings around seeds {research, btech, phd, exam, faculty, institute} and their nearest neighbors were projected to 2D.



(a) t-SNE projection for CBOW embeddings (b) t-SNE projection for Skip-gram embeddings

Figure 2: t-SNE comparison of CBOW and Skip-gram embeddings



(a) PCA projection for CBOW embeddings (b) PCA projection for Skip-gram embeddings

Figure 3: PCA comparison of CBOW and Skip-gram embeddings

Clustering behavior:

- CBOW forms somewhat smoother thematic groupings around institutional and academic terms.
- Skip-gram shows tighter but noisier micro-clusters, including token-fragment clusters influenced by preprocessing or PDF extraction artifacts.

2 Problem 2: Character-Level Name Generation Using RNN Variants

2.1 Objective

To design and compare three recurrent architectures for character-level Indian name generation: Vanilla RNN, BLSTM, and RNN with attention.

2.2 Task 0: Dataset

A training set of 1000 Indian names was generated and stored in `TrainingNames.txt`.

Table 9: Dataset Snapshot for Problem 2

Property	**Value**
Total names	1000
Unique names	1000
Tensorized dataset shape	`[1000, 16]`
Character vocabulary size	27
Special tokens	SOS: `<`, EOS: `>`, PAD: `#`

Global training constants used:

- Embedding dimension = 48
- Hidden dimension = 128
- Number of layers = 2
- Learning rate = 0.002
- Epochs = 50
- Batch size = 64
- Random seed = 42

2.3 Task 1: Model Implementations and Architecture

All three models in Problem 2 were implemented from scratch in PyTorch using custom model classes, explicit forward-pass logic, and a manually written training loop (teacher forcing + cross-entropy objective) for next-character prediction.

2.3.1 Vanilla RNN

Architecture: Embedding → RNN → Linear(vocab).

Trainable parameters:

$$\text{params}_{\text{RNN}} = 60,587$$

Implementation details :

- Built as a custom `nn.Module` with embedding layer, recurrent computation block, and output projection head for vocabulary logits.
- Sequence processing and next-token decoding are handled explicitly in the model forward pass for autoregressive character modeling.
- Hyperparameters used: hidden size = 128, number of layers = 2, learning rate = 0.002.

2.3.2 BLSTM

Architecture: Embedding → Bidirectional LSTM → Linear(vocab).

Trainable parameters:

$$\text{params}_{\text{BLSTM}} = 585,771$$

Implementation details :

- Built as a custom bidirectional LSTM module with explicit concatenation of forward and backward hidden representations before output projection.
- Bidirectional context integration is handled in the model pipeline to exploit both left and right character dependencies.
- Hyperparameters used: hidden size = 128, number of layers = 2, learning rate = 0.002.

2.3.3 RNN with Basic Additive Attention

Architecture: RNN encoder (bidirectional) → additive attention → RNN decoder → Linear(vocab).

Trainable parameters:

$$\text{params}_{\text{Attn}} = 155,307$$

Implementation details :

- Built with a custom encoder-decoder pipeline where additive attention scores, normalization, and context-vector computation are implemented explicitly in code.
- At each decoding step, the attention context is fused with decoder state before projecting to vocabulary logits for next-character prediction.
- Hyperparameters used: hidden size = 128, encoder layers = 2, decoder layers = 1, learning rate = 0.002.

2.3.4 Parameter Comparison

Table 10: Trainable Parameter Comparison

Model	**Trainable Parameters**
Vanilla RNN	60,587
BLSTM	585,771
RNN with Attention	155,307

2.4 Task 2: Quantitative Evaluation

For generated names G and training set T:

$$\text{Novelty Rate} = \frac{|\{g \in G : g \notin T\}|}{|G|}$$

$$\text{Diversity} = \frac{|\text{unique}(G)|}{|G|}$$

Each model was evaluated with 500 generated names.

Table 11: Quantitative Comparison (Problem 2)

Model	**Novelty Rate**	**Diversity**
Vanilla RNN	97.19%	99.40%
BLSTM	100.00%	73.79%
RNN with Attention	75.75%	95.39%

Interpretation:

- BLSTM achieves maximum novelty but reduced diversity, indicating repetitive generation loops.
- Vanilla RNN balances high novelty and high diversity but with less realistic orthography.
- Attention model produces lower novelty (more train-set overlap) but stronger human plausibility in many samples.

2.5 Task 3: Qualitative Analysis

Representative generated samples from notebook outputs:

Table 12: Sample Generated Names

Model	Examples
Vanilla RNN	Ansa, Rashithar, Prayat, Nari, Dildap, Saneravi, Parshita, Areli, Hurina, Saminda
BLSTM	Uuuuuuuuuuuuuuu, Jvegeeeeeeeeeee, Ggggggggggggggg, Uauauauyuyyyyyy, Kakakakakakakak, ...
RNN w/ Attention	Rushita, Sharanaji, Mona, Rathana, Drantadyi, Prajjslal, Bharav, Vilac, Rajkudra, Ishika

2.5.1 Common Failure Modes

- Character repetition loops (most visible in BLSTM outputs).
- Incomplete or awkward suffix construction (e.g., malformed endings).
- Occasional overfitting to high-frequency character n-grams.

2.5.2 Realism Assessment

- **Most realistic overall:** RNN with attention
- **Most novel but unstable:** BLSTM
- **Most diverse string forms:** Vanilla RNN

3 Limitations and Future Improvements

3.1 Observed Limitations

- Problem 1 semantics are affected by mixed-source noise and token artifacts from web/PDF ingestion.
- Problem 2 BLSTM quality suggests mode collapse-like repetition under current generation setup.
- Limited corpus size (640K tokens) for Word2Vec may restrict embedding quality for rare or domain-specific terms; larger institutional corpora would improve statistical significance.
- No out-of-vocabulary (OOV) character handling mechanisms in Problem 2 decoders; would benefit from subword or byte-pair encoding approaches.
- Training epochs (10 for Problem 1, 50 for Problem 2) may be insufficient to capture complex semantic patterns; longer training with proper validation-based early stopping could improve

results.

3.2 Potential Improvements

- Extend Problem 1 to 20-30 epochs and Problem 2 to 100-150 epochs with learning rate scheduling (exponential decay or cosine annealing) to capture deeper semantic patterns and reduce mode collapse in BLSTM generation.
- Add temperature scheduling, top-k/nucleus sampling, and repetition penalties in Problem 2 generation to reduce character loops and improve diversity.
- Implement validation sets and early stopping criteria (e.g., no improvement in validation loss for 10 epochs) to prevent overfitting and improve generalization.
- Incorporate pre-trained embeddings (FastText, GloVe) as baselines to quantify improvement from domain-specific corpus training in Problem 1.
- Experiment with different optimization algorithms (Adam, RMSprop with learning rate warmup) and adaptive scheduling to accelerate convergence in both problems.

4 Conclusion

This assignment successfully implements and compares word representation learning and character-level sequence modeling on institutional data. Problem 1 demonstrates that from-scratch Word2Vec (CBOW and Skip-gram) captures meaningful institutional semantics, with CBOW proving more stable (57–61% analogy accuracy) than Skip-gram under noisy mixed-source corpora. Hyperparameter ablations confirm that optimal performance requires $d = 300$, $w = 5$, $k = 5$. Problem 2 reveals critical architecture trade-offs: BLSTM achieves peak novelty (100%) but suffers mode collapse; Vanilla RNN balances novelty (97.19%) and diversity (99.40%) but lacks orthographic plausibility; RNN with attention produces the most human-like names with 75.75% novelty, demonstrating that attention mechanisms prevent degenerate solutions.